

System Overview

What's built, how it's scoped, who can do what, and what's next

Prepared 2 Sep 2026 · Laravel 11 + React/Vite + PostgreSQL · matches the 21-table class-diagram schema (`database-schema.md`) exactly, column for column

1 What this system is

Ayeyarwady Grill is a hybrid restaurant platform covering two customer journeys and one staff operation: **(a)** a customer books a table online and pays a deposit ahead of time, and **(b)** a customer already seated scans a QR code on their table and orders food directly from their phone — no app install, no waiting for a waiter to take the order. Staff and admins run the kitchen floor, tables, menu, and payments from a dashboard that updates live as orders and bookings come in.

The system is built against a fixed 21-entity database schema (plus Sanctum's own auth token table) that mirrors the project's class diagram. Every feature below was implemented to work *within* that schema — no tables or columns were added to ship any of it.

2 Roles & permissions

Three account types, stored in three separate tables (`customers` , `staff` , `admins`) rather than one shared users table. **Admin inherits every Staff permission** in addition to admin-only screens — there's no separate admin account needed to do what staff do.

ROLE	CAN DO	CANNOT DO
CUSTOMER	Book tables, order via QR, pay deposits, view own history/notifications, edit own profile	See other customers' data, access staff/admin screens
STAFF	Everything a customer-facing floor role needs: live orders, bookings/deposits, own attendance	Manage menu, tables, staff accounts, payment methods, audit log

3 Functionality by role

CUSTOMER

- Register / log in, demo credentials shown right on the login screen
- Browse the public menu by category
- 4-step table booking wizard: date & time → party size/availability → contact details → confirm
- Pay the deposit via a scan-to-pay QR (uploaded by the restaurant) and submit a payment screenshot as proof
- Re-upload a new screenshot if a deposit is rejected
- Dashboard: upcoming reservation, total visits / total spent / member-since stats, recent bookings, and a live notifications panel
- My Bookings — full history with live payment status
- Order History, Notifications (mark as read)
- Edit profile (name, phone, address) — email is intentionally read-only, it's the login identity
- Scan a table's QR to order food directly, with or without an account

STAFF

- Overview dashboard — pending payments, today's bookings, live order count, table occupancy at a glance
- Live Orders — kitchen queue, advance each order's status (placed → preparing → ready → served → completed)
- Bookings & Deposits — see every booking, view a customer's uploaded payment screenshot, confirm or reject the deposit against the restaurant's own KBZPay records
- Attendance — clock in/out, view own history

ADMIN

(everything Staff has, plus:)

- Manage Menu — categories and items, availability toggle, category visibility toggle
- Tables & QR — add tables, generate/regenerate each table's QR code, manual status override
- Staff Accounts — create accounts, activate/deactivate (history is preserved, never deleted)
- Payment Methods — add methods (KBZPay/Cash/Card/...), activate/deactivate, upload the actual scan-to-pay QR image customers see at checkout
- Audit Log — every admin create/update/delete action, with before/after values; individual entries can be deleted

4 Notable system behavior

Table status automation

Tables move through their lifecycle automatically, not just via manual staff toggles:

Available → customer scans QR → **Reserved**

Reserved or **Available** → order placed →

Occupied

Reserved, no order in 20 min → **Available** again

Re-scanning an already-Reserved table does not restart its 20-minute window

Deposit / payment verification flow

A booking is created `PENDING` with a `PENDING` deposit — it is *not* auto-confirmed. The customer pays externally and uploads a screenshot; staff review it against their own KBZPay dashboard and confirm or reject. Confirming flips the booking to `CONFIRMED` and logs the transition in `booking_status_logs`; the screenshot itself lives on disk (not in the database) with only its path referenced from the existing `payments.notes` column — no schema change was needed to support it.

Real-time updates

Live order/booking/table-status pushes run over Laravel Reverb (WebSockets). Every broadcast is wrapped so a broadcast failure never blocks the underlying save — worst case, a dashboard misses an instant push and shows the correct state on next refresh.

5 Known scope simplifications

AREA	CURRENT STATE
Order payments	Only booking deposits are tracked as real <code>Payment</code> rows today; orders placed via QR/walk-in don't yet create one.
Menu item images	Stored as an image URL, not a file upload.
Email address	Fixed after registration (it's the login identity) — no self-service change or verification flow yet.
Reservation timeout	Checked lazily on read (table list / QR scan), not by a background scheduler — accurate whenever the app is being used, not to the exact second if left completely idle.
Payments	Fully manual verification against screenshots — no live payment gateway integration.

6 Planned / future features

- Editable customer email with uniqueness validation (currently read-only by design)
- Real payment gateway integration (KBZPay/WavePay API) to replace manual screenshot verification
- Order-level payment records, so dine-in/QR orders are tracked the same way booking deposits are
- Scheduled (cron-based) reservation expiry for exact 20-minute timing even with no one viewing the app

- Email/SMS notifications, in addition to the current in-app notification center
- File-upload menu item images (replacing the URL-only field)
- Analytics/reporting dashboard — revenue trends, popular items, booking patterns over time
- Customer email verification and self-service password reset